Cyber Security

API SECURITY RISK ANALYSIS REPORT (Task 3)

API Tested: JSONPlaceholder API (https://jsonplaceholder.typicode.com/comments)
Date: May 4, 2026
By: Hanoof Nadeem
Tools Used: Browser DevTools, Manual API Testing

EXECUTIVE SUMMARY:

An API security risk analysis was performed on the JSONPlaceholder API, focusing on the /comments endpoint. Four security risks were identified: 2 High severity, 1 Medium severity, and 1 Low severity.

Most Critical Issues:

1, No authentication required - Anyone can access all data without an API key (HIGH)

2, ID enumeration - Users can access any comment by changing ID numbers in URL (HIGH)

If this were a real production API:

- Attackers could steal all customer data
- Personal information (names, emails) would be exposed
- The API could be abused for denial of service attacks

Overall Risk Score: 7/10 (High risk due to missing authentication)

INTRODUCTION :

APIs power modern applications like mobile apps, websites, and software integrations. Insecure APIs can expose sensitive data, allow unauthorized access, and lead to data breaches.

This report analyzes the JSONPlaceholder API (/comments endpoint) to identify common security weaknesses. The assessment used browser DevTools and manual endpoint testing. All testing was read-only with no exploitation.

Why This Matters:

- APIs are everywhere in modern software
- Insecure APIs cost businesses millions in data breaches
- API security is a critical skill for security professionals

Methodology :

STEPS	ACTION
1	Open browser DevTools to monitor network requests
2	Access /comments endpoint to check authentication requirements
3	Access /comments/1, /comments/2, /comments/3 to test ID enumeration
4	Inspect response data for excessive exposure
5	Document all findings with severity classification

Tools Used :

Tool	Purpose
Browser DevTools (F12)	Inspect API requests, headers, and responses
Manual endpoint testing	Verify enumeration vulnerability
JSONPlaceholder API	Target for security analysis

RISK FINDINGS :

– FINDING 1: No Authentication Required

What Happened:

When I visited “ https://jsonplaceholder.typicode.com/comments “, the API returned all 500 comments. It did not ask for a password, API key, or any login.

```
  {
    "postId": 1,
    "id": 3,
    "name": "odio adipisci rerum aut animi",
    "email": "Nikita@garfield.biz",
    "body": "quia molestiae reprehenderit quasi aspernatur\naut expedita occaecati aliquam eveniet laudantium\nomnis quibusdam delectus
saepe quia accusamus maiores nam est\ncum et ducimus et vero voluptates excepturi deleniti ratione"
  },
  {
    "postId": 1,
    "id": 4,
    "name": "alias odio sit",
    "email": "Lew@alysha.tv",
    "body": "non et atque\noccaecati deserunt quas accusantium unde odit nobis qui voluptatem\nquia voluptas consequuntur itaque
dolor\net qui rerum deleniti ut occaecati"
  },
  {
    "postId": 1,
    "id": 5,
    "name": "vero eaque aliquid doloribus et culpa",
    "email": "Hayden@althea.biz",
    "body": "harum non quasi et ratione\ntempore iure ex voluptates in ratione\nharum architecto fugit inventore cupiditate\nvoluptates
magni quo et"
  },
  {
    "postId": 2,
    "id": 6,
    "name": "et fugit eligendi deleniti quidem qui sint nihil autem",
    "email": "Presley.Mueller@myrl.com",
    "body": "doloribus at sed quis culpa deserunt consectetur qui praesentium\naccusamus fugiat dicta\nvoluptatem rerum ut voluptate
autem\nvoluptatem repellendus aspernatur dolorem in"
  },
  {
```


Severity: HIGH

Why HIGH:

- No barrier to entry - completely open
- Cannot track or block malicious users
- If this were real customer data, it would all be exposed

– FINDING 2: Excessive Data Exposure


```
[
  {
    "postId": 1,
    "id": 1,
    "name": "id labore ex et quam laborum",
    "email": "Eliseo@gardner.biz",
    "body": "laudantium enim quasi est quidem magnam voluptate ipsam
et nam sapiente accusantium"
  },
  {
    "postId": 1,
    "id": 2,
    "name": "quo vero reiciendis velit similique earum",
    "email": "Jayne_Kuhic@sydney.com",
    "body": "est natus enim nihil est dolore omnis voluptatem numqua
pariatur\nnihil sint nostrum voluptatem reiciendis et"
  },
  {
    "postId": 1,
    "id": 3,
    "name": "odio adipisci rerum aut animi",
    "email": "Nikita@garfield.biz",
    "body": "quia molestiae reprehenderit quasi aspernatur\naut expe
saepe quia accusamus maiores nam est\ncum et ducimus et vero volupta
  },
  {
    "postId": 1,
    "id": 4,
    "name": "alias odio sit",
    "email": "Lew@alysha.tv",
    "body": "non et atque\noccaecati deserunt quas accusantium unde
dolor\net qui rerum deleniti ut occaecati"
  },
  {
    "postId": 1,
    "id": 5,
    "name": "vero eaque aliquid doloribus et culpa",
    "email": "Hayden@althea.biz",
    "body": "harum non quasi et ratione\ntempore iure ex voluptates
magni quo et"
  },
```

Severity: MEDIUM

Why MEDIUM:

- Not critical but still a privacy concern
- Increases bandwidth usage
- Could violate data protection laws

– FINDING 3: ID Enumeration


```
{
  "postId": 1,
  "id": 1,
  "name": "id labore ex et quam laborum",
  "email": "Eliseo@gardner.biz",
  "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis
et nam sapiente accusantium"
}
```

```
{
  "postId": 1,
  "id": 2,
  "name": "quo vero reiciendis velit similique earum",
  "email": "Jayne_Kuhic@sydney.com",
  "body": "est natus enim nihil est dolore omnis voluptatem numquam\net omnis occaecati quod ullam at\nvoluptatem error expedita
pariatur\nnihil sint nostrum voluptatem reiciendis et"
}
```

```
{
  "postId": 1,
  "id": 3,
  "name": "odio adipisci rerum aut animi",
  "email": "Nikita@garfield.biz",
  "body": "quia molestiae reprehenderit quasi aspernatur\naut expedita occaecati aliquam eveniet laudantium\nomnis quibusdam delectus
saepe quia accusamus maiores nam est\ncum et ducimus et vero voluptates excepturi deleniti ratione"
}
```


What Happened:

I accessed:

- **/comments/1 –** got comment with ID 1
- **/comments/2** – got comment with ID 2
- **/comments/3 –** got comment with ID 3

By simply changing the number in the URL, I could access different records.

Severity: HIGH

Why HIGH:

- Easy to automate (just change a number)
- No limit on how many IDs you can try
- Entire database can be stolen quickly

– FINDING 5: Missing Security Headers (LOW RISK)

Headers | Preview | Response | Initiator | Timing

General

Request URL	https://jsonplaceholder.typicode.com/comments
Request Method	GET
Status Code	200 OK
Remote Address	104.21.59.19:443
Referrer Policy	strict-origin-when-cross-origin

Access-Control-Allow-Credentials	true
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=43200
Cf-Cache-Status	MISS
Cf-Ray	9f677c37ac9e141f-MRS
Content-Encoding	zstd
Content-Type	application/json; charset=utf-8
Date	Mon, 04 May 2026 12:24:56 GMT
Etag	W/"154-Geb7blpVmQ1TgQle+Pxz3R7cpJg"
Expires	-1
Nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Pragma	no-cache
Priority	u=0,i
Report-To	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=CI5L81MsPCLpErNO%2BezGVM9Oe4FAJfIXozqAs%2FPzGgU%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1777897496"}],"max_age":3600}
Reporting-Endpoints	heroku-nel="https://nel.heroku.com/reports?s=CI5L81MsPCLpErNO%2BezGVM9Oe4FAJfIXozqAs%2FPzGgU%3D&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1777897496"
Server	cloudflare
Server-Timing	cfCacheStatus;desc="MISS"
Server-Timing	cfEdge;dur=10,cfOrigin;dur=285
Server-Timing	cfExtPri
Vary	Origin, Accept-Encoding
Via	2.0 heroku-router
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-Ratelimit-Limit	1000
X-Ratelimit-Remaining	999
X-Ratelimit-Reset	1777897519

Request Headers

:authority	jsonplaceholder.typicode.com
:method	GET
:path	/comments/3
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.9,ar;q=0.8
Priority	u=0, i
Sec-Ch-Ua	"Google Chrome";v="147", "Not.A/Brand";v="8", "Chromium";v="147"
Sec-Ch-Ua-Mobile	?1
Sec-Ch-Ua-Platform	"Android"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1
Upgrade-Insecure-Requests	1
User-Agent	Mozilla/5.0 (Linux; Android 6.0; Nexus 5 Build/MRA58N) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Mobile Safari/537.36

What is Present (Good):

- X-Content-Type-Options: nosniff - prevents MIME sniffing attacks
- X-Ratelimit-Limit: 1000 - rate limiting is enabled
- X-Ratelimit-Remaining: 999 - tracks remaining requests
- Cache-Control - proper caching configured
- Content-Type: application/json - correct content type

What is Missing:

- Content-Security-Policy (CSP) - helps prevent XSS attacks
- Strict-Transport-Security (HSTS) - forces browsers to use HTTPS
- Referrer-Policy - controls privacy of referrer information
- X-Frame-Options - prevents clickjacking attacks

Severity: LOW

Why LOW:

- Does not directly expose data
- Hardening issue, not a critical vulnerability
- API still has basic protections (rate limiting, MIME sniffing prevention)
- Easy to fix

Business Impact :

– Finding 1: No Authentication Required

What this means for the business:

Your customer data is publicly available. Anyone with an internet connection can access it. There is no lock on the door.

Real-world consequences:

Consequences	Explanation
Data theft	Competitors can steal your entire customer database in minutes
Privacy violation	Customer names, emails, and personal information are exposed to strangers
No accountability	You cannot track who accessed what data or when
Legal liability	GDPR, CCPA, and other privacy laws require data protection. Fines can reach millions of dollars
Loss of customer trust	Customers will leave if they discover their data is public

Example: If this were a banking API, any hacker could see all customer account balances and personal details.

Business cost: Potential loss of customers + regulatory fines + reputational damage

– Finding 2: ID Enumeration

What this means for the business:

Attackers can guess your data IDs because they are simple numbers (1, 2, 3...). They can write a script to download everything in seconds.

Real-world consequences:

Consequences	Explanation
Mass data theft	A simple script can loop through IDs 1 to 1,000,000 and steal everything
No protection	There is no limit on how many IDs an attacker can try
Auotmated Attacks	Attackers do not need special skills - basic scripts work
Complete exposure	Every single record in your database can be stolen

Example: If your customer IDs are 1001, 1002, 1003..., an attacker tries 1001, 1002, 1003... and gets every customer's data.

Business cost: Complete database breach + all customer data exposed + massive legal liability

– Finding 3: Excessive Data Exposure

What this means for the business:

Your API is giving out more information than necessary. Even if a customer only wants to see a post title, you are also sending their email address.

Real-world consequences:

Consequences	Explanation
Privacy Risk	Email addresses are exposed even when not needed
GDPR Violation	Processing more personal data than necessary violates data minimization principles
Increased attack surface	More data exposed means more potential for misuse
Wasted bandwidth	Slower responses and higher server costs

Example: A mobile app only needs to show comment text, but the API also sends user email addresses to everyone.

Business cost: Regulatory fines + customer privacy concerns + slower application performance.

– Finding 4: Missing Security Headers

What this means for the business:

Your API lacks additional security protections that modern browsers expect. While not an immediate data breach risk, it shows incomplete security hardening.

Real-world consequences:

Consequences	Explanation
XSS Risk	Without CSP, API documentation pages could be vulnerable to script injection
Protocol downgrade	Without HSTS, attackers could force connections to use insecure HTTP
Information leak	Without Referrer-Policy, sensitive URLs could leak to other sites
Professional Standard	Most production APIs implement these headers as baseline security

Example: An attacker could create a fake API documentation page that looks identical to yours and steal developer credentials.

Business cost: Low immediate risk but indicates incomplete security posture

REMEDIATION SUGGESTIONS :

– Finding 1: No Authentication Required

What is the problem?

Anyone can access your API without a password, API key, or any form of identification.

The Fix:

Action	What to do	Difficulty
Implement API keys	Require a unique API key for every request	Easy
Use OAuth 2.0	Allow users to log in with existing accounts (Google, Facebook, etc.)	Medium
Add token authentication	Require a JWT (JSON Web Token) with every request	Medium

Technical steps for developers:

1. Generate unique API keys for each customer
2. Reject all requests without a valid key
3. Log all authentication attempts for monitoring
4. Implement key rotation every 90 days

Timeframe: 24 hours (critical priority)

– Finding 2: ID Enumeration

What is the problem?

Users can guess data IDs because they are simple numbers (1, 2, 3...). Attackers can change the number and access any record.

The Fix:

Action	What to do	Difficulty
Use UUIDs instead of numbers	Replace IDs like 1,2,3 with random codes like: f47ac10b-58cc-4372-a567-0e02b2c3d479	Easy
Add authorization checks	Verify the user has permission to access each requested record	Medium
Implement rate limiting	Limit how many IDs an attacker can try (already present - 1000 requests)	Already d

Technical steps for developers:

1. Change database ID column from integer to UUID
2. Update API endpoints to accept UUIDs instead of numbers
3. Add authorization check: "Does the logged-in user own this record?"
4. Return 404 or 403 for unauthorized access attempts

Timeframe: 1 week (high priority)

– Finding 3: Excessive Data Exposure

What is the problem?

The API returns more data than needed. For example, returning email addresses when only names are requested.

The Fix:

Action	What to do	Difficulty
Add field filtering	Allow clients to request specific fields: ?fields=id,name	Easy
Review API responses	Remove fields that are not absolutely necessary	Easy
Use GraphQL	Let clients ask for exactly what they need	Medium

Technical steps for developers:

1. Audit each API endpoint and remove unnecessary fields
2. Implement field filtering using query parameters
3. Create different response schemas for different use cases
4. Apply data minimization principles from GDPR

Timeframe: 2 weeks

– Finding 4: Missing Security Headers

What is the problem?

The API response is missing important security headers that help protect browsers and clients.

The Fix:

Add these headers to all API responses:

Header	Value	What it does
Content-Security-Policy	default-src 'none'	Prevents XSS attacks
Strict-Transport-Security	max-age=31536000	Forces HTTPS connections
Referrer-Policy	no-referrer	Prevents URL leakage
X-Frame-Options	DENY	Prevents clickjacking

Technical steps for developers:

1. Add headers to server configuration (Nginx/Apache/Express)
2. Test headers are present using browser DevTools
3. Set CSP to restrictive policy first, then relax if needed
4. Enable HSTS preload for additional security

Timeframe: 1 month (low priority)